

**SISTEM KASIR DAN MANAJEMEN ANTREAN  
RESTORAN BERBASIS COMMAND LINE INTERFACE  
MENGUNAKAN BAHASA PYTHON**



Disusun untuk memenuhi tugas akhir Project-Based Learning  
(PBL) Mata Kuliah Algoritma & Struktur Data (TPL2106)

Disusun oleh:  
**Kelompok 3**

|                           |                    |
|---------------------------|--------------------|
| <b>Fardhan Al Biansya</b> | <b>J0403251060</b> |
| <b>Hirzi Haidi Haqqi</b>  | <b>J0403251073</b> |
| <b>Dawam Raihan Zufar</b> | <b>J0403251099</b> |

Program Studi Teknologi Rekayasa Perangkat Lunak  
Sekolah Vokasi IPB University

## KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan akhir *Project-Based Learning* (PBL) yang berjudul “Sistem Kasir Dan Manajemen Antrean Restoran Berbasis *Command Line Interface* Dengan Bahasa Python” ini dapat diselesaikan dengan baik. Laporan ini disusun untuk memenuhi tugas akhir mata kuliah Algoritma dan Struktur Data pada Program Studi Teknologi Rekayasa Perangkat Lunak, Sekolah Vokasi IPB University.

Penulis mengucapkan terima kasih kepada Ibu Dr. Shelve Nidya Neyman S.Kom., M.Si. dan Ibu Dr. Sofiyanti Indriasari S.Kom., M.Kom. selaku dosen pengampu mata kuliah Algoritma dan Struktur Data yang telah memberikan bimbingan, arahan, serta ilmu yang sangat bermanfaat selama proses perkuliahan maupun penyusunan proyek ini.

Penulis menyadari bahwa baik aplikasi yang dibangun maupun laporan ini masih memiliki banyak kekurangan dan jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi perbaikan pada kesempatan berikutnya. Semoga laporan ini dapat memberikan manfaat bagi penulis dan pembaca, dalam memahami penerapan struktur data *Single Linked List* dan *Queue* pada penyelesaian permasalahan nyata.

Bogor, 22 Juni 2026

Kelompok 3

## DAFTAR ISI

|                                               |           |
|-----------------------------------------------|-----------|
| <b>KATA PENGANTAR .....</b>                   | <b>2</b>  |
| <b>BAB I. PENDAHULUAN .....</b>               | <b>5</b>  |
| 1.1 Latar Belakang .....                      | 5         |
| 1.2 Tujuan Proyek .....                       | 6         |
| 1.3 Ruang Lingkup .....                       | 6         |
| <b>BAB II. ANALISIS DAN PERANCANGAN .....</b> | <b>7</b>  |
| 2.1 Deskripsi Sistem .....                    | 7         |
| 2.2 Struktur Data yang Digunakan .....        | 7         |
| 2.3 Algoritma yang Digunakan .....            | 8         |
| 2.4 Diagram Alir Program (Flowchart) .....    | 10        |
| 2.5 Struktur Program .....                    | 10        |
| <b>BAB III. IMPLEMENTASI PROGRAM .....</b>    | <b>12</b> |
| 3.1 Tampilan Menu Utama .....                 | 12        |
| 3.2 Fitur Create .....                        | 13        |
| 3.3 Fitur Read .....                          | 14        |
| 3.4 Fitur Update .....                        | 15        |
| 3.5 Fitur Delete .....                        | 16        |
| 3.6 Fitur Searching .....                     | 16        |
| 3.7 Fitur Sorting .....                       | 17        |
| 3.8 Fitur Antrean Pelanggan (Queue) .....     | 18        |
| 3.9 Fitur Transaksi Kasir .....               | 19        |
| 3.10 Penyimpanan Data (File Handling) .....   | 20        |
| 3.11 Validasi Input dan Error Handling .....  | 21        |
| <b>BAB IV. PENGUJIAN DAN ANALISIS .....</b>   | <b>22</b> |
| 4.1 Skenario Pengujian .....                  | 22        |
| 4.2 Kelebihan Sistem .....                    | 23        |

|                                              |           |
|----------------------------------------------|-----------|
| 4.4 Keterbatasan Sistem .....                | 23        |
| <b>BAB V. KENDALA DAN SOLUSI.....</b>        | <b>24</b> |
| <b>BAB VI. PEMBAGIAN TUGAS ANGGOTA .....</b> | <b>25</b> |
| <b>BAB VII. KESIMPULAN DAN SARAN .....</b>   | <b>26</b> |
| <b>DAFTAR PUSTAKA .....</b>                  | <b>27</b> |
| <b>LAMPIRAN.....</b>                         | <b>28</b> |
| Lampiran A. Link GitHub.....                 | 28        |
| Lampiran B. Screenshot Program.....          | 28        |
| Lampiran C. Struktur Folder Proyek.....      | 28        |
| Lampiran D. Commit History GitHub .....      | 28        |
| Lampiran E. Source Code Utama.....           | 29        |

# BAB I

## PENDAHULUAN

### 1.1 Latar Belakang

Pengelolaan operasional restoran skala kecil hingga menengah pada umumnya masih dilakukan secara manual, mulai dari pencatatan menu, penentuan urutan pelanggan yang dilayani, hingga pencatatan transaksi penjualan. Pendekatan manual tersebut berpotensi besar terhadap berbagai permasalahan, seperti kesalahan pencatatan harga, ketidaksesuaian data stok, serta tidak adanya mekanisme yang adil dalam menentukan urutan pelanggan yang harus dilayani terlebih dahulu. Ketika jumlah pelanggan restoran semakin ramai, pencatatan manual menjadi semakin sulit untuk dikelola secara konsisten dan memungkinkan untuk menimbulkan kekeliruan.

Berdasarkan permasalahan tersebut, kelompok kami merancang dan membuat sebuah aplikasi kasir berbasis *Command Line Interface* (CLI) bernama SARIAWAM (Restoran Sariawam) menggunakan bahasa pemrograman Python. Aplikasi ini menerapkan dua struktur data fundamental yang dipelajari pada mata kuliah Algoritma dan Struktur Data, yaitu *Single Linked List* untuk mengelola data menu secara dinamis, dan *Queue* (antrean *First In First Out*/FIFO) untuk mengatur urutan pelanggan yang akan dilayani di kasir. Dengan penerapan kedua struktur data tersebut, prinsip “pelanggan yang datang lebih dahulu dilayani lebih dahulu” dapat diterapkan secara konsisten. Pengelolaan data menu seperti penambahan, pengubahan, penghapusan, pencarian, dan pengurutan harga juga dapat dilakukan secara terprogram dan efisien.

Selain itu, aplikasi ini juga dilengkapi dengan mekanisme penyimpanan data permanen menggunakan berkas teks (data/menu.txt) sehingga data menu tidak akan hilang ketika program dihentikan dan dijalankan kembali. Tampilan antarmuka *command line* juga dipercantik menggunakan pustaka pihak ketiga rich, sehingga interaksi pengguna dengan program menjadi lebih nyaman dibandingkan tampilan CLI konvensional.

## 1.2 Tujuan Proyek

1. Membuat aplikasi kasir restoran berbasis CLI/terminal menggunakan bahasa pemrograman Python.
2. Mengimplementasikan struktur data *Single Linked List* untuk pengelolaan data menu secara dinamis (*Create, Read, Update, Delete*).
3. Mengimplementasikan struktur data *Queue* (FIFO) untuk mengelola antrean pelanggan secara adil dan terurut.
4. Menerapkan algoritma pencarian (*searching*) dan pengurutan (*sorting*) pada data menu restoran.
5. Menerapkan konsep *file handling* agar data menu tersimpan secara permanen pada media penyimpanan.
6. Mengembangkan solusi atas permasalahan nyata dalam pengelolaan kasir dan antrean restoran skala kecil hingga menengah.

## 1.3 Ruang Lingkup

- Platform: *Command Line Interface* (CLI) berbasis terminal.
- Bahasa pemrograman: Python 3, dengan pustaka pihak ketiga rich untuk antarmuka.
- Penyimpanan data: berkas teks (.txt) dengan format menyerupai CSV (nilai dipisahkan koma).
- Fitur CRUD data menu melalui mode admin tersembunyi (kode akses '67').
- Fitur pencarian menu (*sequential search*, pencarian sebagian/*partial match*).
- Fitur pengurutan harga menu (*bubble sort*, *ascending* dan *descending*).
- Fitur manajemen antrean pelanggan (*enqueue* dan *dequeue*, FIFO) untuk tipe *Dine-in* dan *Takeaway*.
- Fitur transaksi kasir meliputi pemilihan menu, perhitungan total belanja, pengurangan stok otomatis, dan pencetakan struk pembelian.

## BAB II

### ANALISIS DAN PERANCANGAN

#### 2.1 Deskripsi Sistem

Sistem SARIAWAM merupakan aplikasi kasir restoran berbasis CLI yang terdiri atas dua modul struktur data utama, yaitu modul *LinkedListMenu* yang menangani seluruh data menu makanan dan minuman, serta modul *QueuePesanan* yang menangani antrean pelanggan beserta proses transaksi di kasir. Saat aplikasi dijalankan, data menu pada berkas *data/menu.txt* dimuat ke dalam *Single Linked List* di memori melalui method *muat\_dari\_txt()*. Pengguna kemudian dapat berinteraksi dengan sistem melalui lima layanan utama pada menu interaktif, yaitu menampilkan dan mencari menu, mengambil nomor antrean, menampilkan status antrean, melakukan transaksi di kasir, serta keluar dari aplikasi.

Selain kelima layanan tersebut, tersedia pula mode admin tersembunyi yang dapat diakses melalui kode rahasia '67' pada menu utama, yang memungkinkan pengelola restoran melakukan operasi *Create*, *Update*, dan *Delete* terhadap data menu. Setiap perubahan data menu yang dilakukan baik melalui mode admin maupun melalui proses transaksi (pengurangan stok) akan langsung disesuaikan ke dalam berkas *data/menu.txt* melalui method *simpan\_data\_ke\_txt()* sehingga data tetap konsisten dan tidak hilang ketika program dihentikan.

#### 2.2 Struktur Data yang Digunakan

| Struktur Data      | Fungsi                                                                                                                                                     |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Single Linked List | Menyimpan dan mengelola seluruh data menu restoran (jenis, nama, harga, stok) secara dinamis termasuk operasi tambah, ubah, hapus, cari, dan urutkan menu. |
| Queue (FIFO)       | Mengelola antrean pelanggan yang mengambil nomor antrean ( <i>Dine-in/Takeaway</i> ) dan memastikan pelanggan dilayani sesuai urutan kedatangan di kasir.  |

Pada proyek ini, struktur data *Stack* dan *Dictionary* tidak digunakan secara eksplisit karena seluruh kebutuhan fungsional sistem baik pengelolaan menu maupun antrean pelanggan dapat dipenuhi secara optimal melalui kombinasi *Single Linked List* dan *Queue*. Pengelompokan menu berdasarkan kategori pada fitur tampilan menu memang memanfaatkan dictionary bawaan Python sebagai struktur bantu sementara, namun bukan sebagai struktur data penyimpanan utama.

### 2.2.1 Alasan Pemilihan Struktur Data

*Single Linked List* dipilih untuk menyimpan data menu karena jumlah menu bersifat dinamis, yaitu dapat bertambah maupun berkurang kapan saja melalui mode admin. Dibandingkan *array* atau *list statis*, *Single Linked List* memudahkan proses penyisipan data baru di bagian akhir (method `tambah_di_akhir()`) maupun penghapusan node tertentu tanpa perlu menggeser seluruh elemen lain cukup dengan mengubah pointer *next* pada node sebelumnya.

*Queue* dipilih untuk mengelola antrean pelanggan karena mengikuti prinsip *First In First Out* (FIFO), yaitu pelanggan yang lebih dahulu mengambil nomor antrean akan lebih dahulu dilayani di kasir. Implementasi pada proyek ini menggunakan struktur list bawaan Python, dengan operasi *enqueue* melalui method `append()` pada saat pelanggan mengambil nomor antrean, dan operasi *dequeue* melalui method `pop(0)` pada saat pelanggan tersebut selesai bertransaksi di kasir.

## 2.3 Algoritma yang Digunakan

### 2.3.1 Searching

Algoritma yang digunakan adalah *Sequential Search* (pencarian berurutan). Algoritma tersebut diterapkan pada method `cari_menu()` untuk pencarian berdasarkan nama persis (*exact match*, digunakan saat admin melakukan *update* atau *delete* maupun saat kasir mencatat pesanan), dan method `cari_menu_sebagian()` untuk pencarian berdasarkan kata kunci sebagian (*partial match*, digunakan pada fitur pencarian menu oleh pelanggan).



Kedua method tersebut melakukan penelusuran node demi node dari head hingga akhir *Single Linked List* lalu membandingkan nama menu (yang telah diseragamkan ke huruf kecil agar case-insensitive) dengan kata kunci pencarian.

Sequential Search dipilih karena struktur data yang digunakan adalah *Single Linked List*, yang hanya dapat diakses secara berurutan (*sequential access*) dan tidak mendukung pengaksesan acak (*random access*) seperti pada *array* sehingga algoritma pencarian biner (*binary search*) tidak dapat diterapkan secara langsung pada struktur ini. Selain itu, jumlah data menu pada proyek ini relatif kecil (kurang dari 50 item pada data awal), sehingga kompleksitas  $O(n)$  pada *Sequential Search* masih sangat efisien digunakan tanpa berdampak signifikan terhadap performa aplikasi.

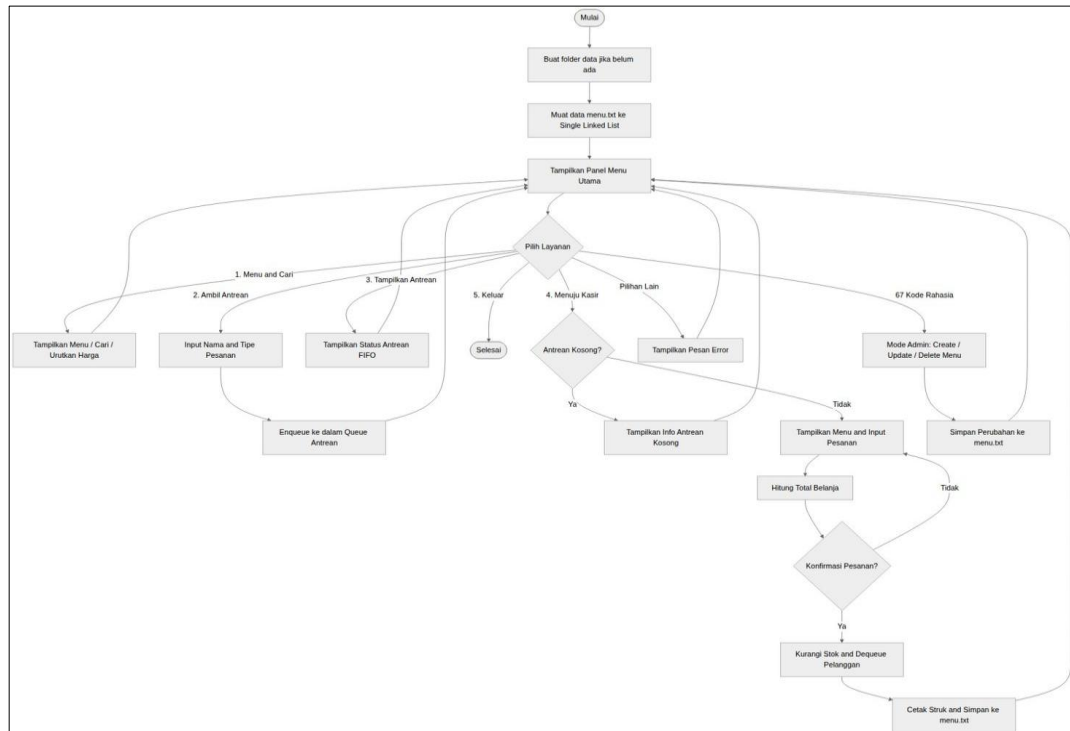
### 2.3.2 Sorting

Algoritma pengurutan yang digunakan adalah *Bubble Sort*. Algoritma tersebut diterapkan pada method `urutkan_harga(ascending)` untuk mengurutkan data menu berdasarkan harga baik secara *ascending* (termurah ke termahal) maupun *descending* (termahal ke termurah). Karena struktur data berupa *Single Linked List*, proses pengurutan tidak menukar pointer antar node (yang berisiko merusak rantai *linked list* apabila ditangani secara sembarangan), melainkan menukar isi atau *value* dari setiap atribut node (jenis, nama, harga, stok) ketika dua node yang berdekatan tidak berada pada urutan yang benar.

*Bubble Sort* dipilih karena kesederhanaan logikanya yang mudah diimplementasikan dan dipahami pada struktur *Single Linked List*. Proses pertukaran cukup dilakukan terhadap node yang saling bersebelahan (`current` dan `current.next`) tanpa memerlukan akses indeks acak. Meskipun memiliki kompleksitas waktu  $O(n^2)$  pada kasus terburuk yang relatif kurang efisien dibandingkan algoritma *sorting* lain seperti *Merge Sort*, hal ini dapat diterima mengingat jumlah data menu pada proyek ini tergolong kecil (38 item pada data awal) sehingga dampak terhadap performa program secara keseluruhan tidak signifikan.

## 2.4 Diagram Alir Program (Flowchart)

Diagram alir berikut menggambarkan keseluruhan alur eksekusi program SARIAWAM, mulai dari pemuatan data, penyajian menu layanan utama, hingga proses pada masing-masing layanan termasuk mode admin.



Gambar 2.1 Diagram alir (flowchart) program SARIAWAM

## 2.5 Struktur Program

Proyek ini disusun dengan struktur berkas yang modular agar memudahkan pengembangan dan pemeliharaan, dengan rincian sebagai berikut.

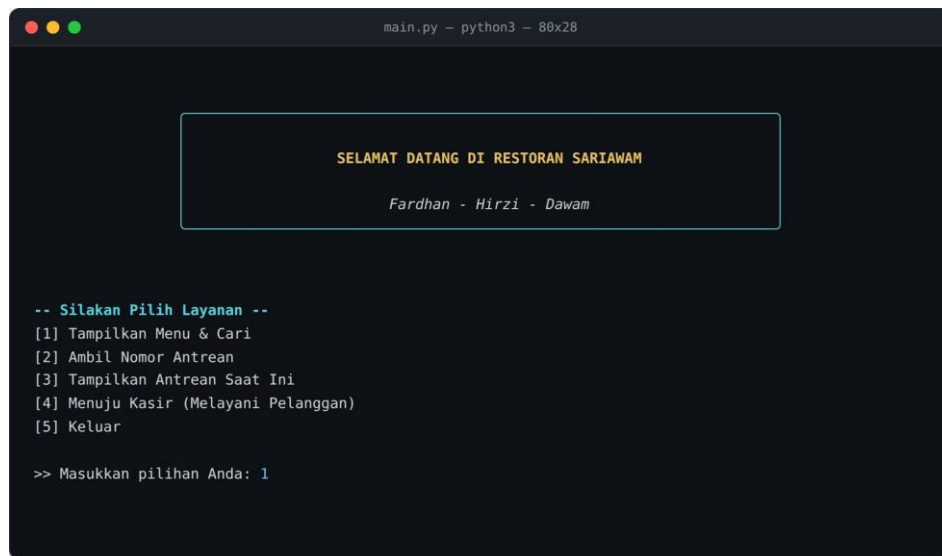
| Berkas             | Deskripsi                                                                                                                                         |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| main.py            | Berisi fungsi main() yang menjadi titik masuk ( <i>entry point</i> ) program, mengatur seluruh alur menu interaktif serta mode admin (kode '67'). |
| linkedlist_menu.py | Berisi class NodeMenu dan LinkedListMenu yang mengelola seluruh operasi terhadap data menu menggunakan Single Linked List.                        |

| Berkas           | Deskripsi                                                                                                                     |
|------------------|-------------------------------------------------------------------------------------------------------------------------------|
| queue_antrean.py | Berisi class QueuePesanan yang mengelola antrean pelanggan (Queue), proses transaksi kasir, serta pencetakan struk pembelian. |
| menu.txt         | Berkas penyimpanan data menu secara permanen, berformat teks dengan nilai dipisahkan koma (jenis, nama, harga, stok).         |
| README.md        | Dokumentasi proyek, termasuk daftar fitur, struktur direktori, dan panduan instalasi/menjalankan aplikasi.                    |

## BAB III

# IMPLEMENTASI PROGRAM

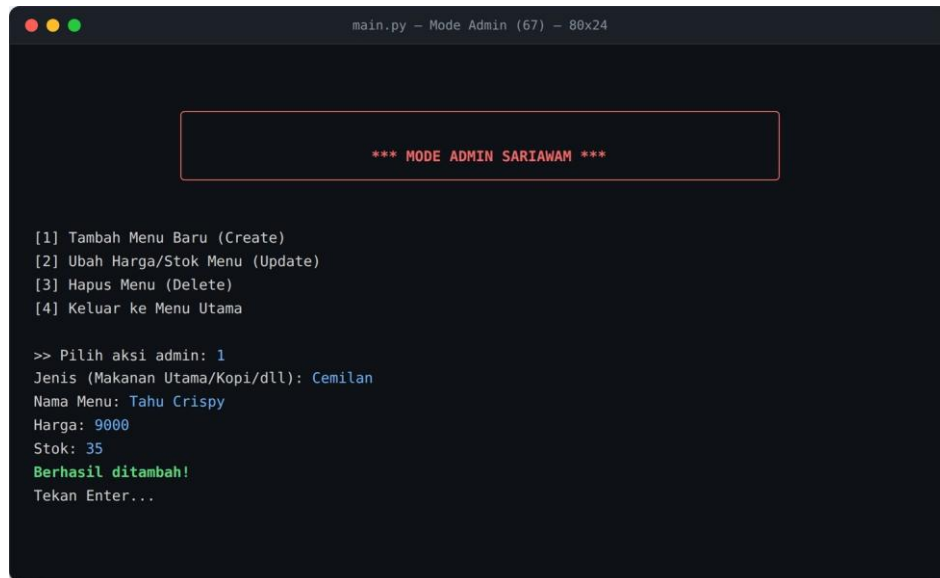
### 3.1 Tampilan Menu Utama



Gambar 3.1 Tampilan menu utama aplikasi SARIWAM

Pada Gambar 3.1 ditampilkan tampilan awal aplikasi SARIWAM yang dibangun menggunakan pustaka rich khususnya komponen *Panel* dan *Align* untuk menampilkan judul aplikasi secara terpusat dengan kombinasi warna cyan, kuning, dan putih. Pengguna dapat memilih satu dari lima layanan utama, yaitu menampilkan atau mencari menu, mengambil nomor antrean, menampilkan status antrean, menuju kasir, atau keluar dari aplikasi, melalui fungsi `main()` pada berkas `main.py`.

## 3.2 Fitur Create



```
main.py - Mode Admin (67) - 80x24

*** MODE ADMIN SARIAWAM ***

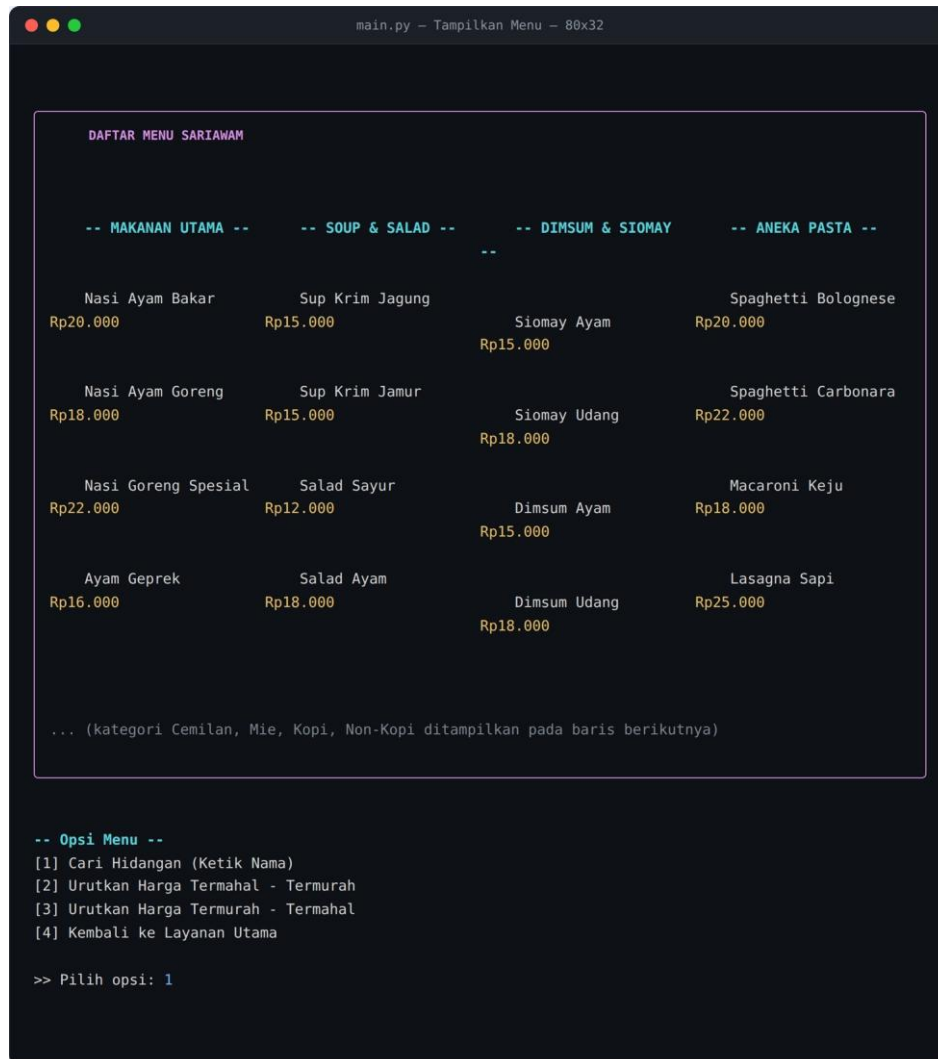
[1] Tambah Menu Baru (Create)
[2] Ubah Harga/Stok Menu (Update)
[3] Hapus Menu (Delete)
[4] Keluar ke Menu Utama

>> Pilih aksi admin: 1
Jenis (Makanan Utama/Kopi/dll): Cemilan
Nama Menu: Tahu Crispy
Harga: 9000
Stok: 35
Berhasil ditambah!
Tekan Enter...
```

Gambar 3.2 Fitur Create — Tambah Menu Baru pada mode admin

Fitur penambahan menu baru (Create) hanya dapat diakses melalui mode admin yang tersembunyi, yaitu dengan memasukkan kode rahasia ‘67’ pada menu layanan utama. Admin diminta memasukkan jenis, nama, harga, dan stok menu baru, kemudian data tersebut ditambahkan ke akhir *Single Linked List* melalui method `tambah_di_akhir()` pada class `LinkedListMenu`. Proses ini melakukan traversal hingga node terakhir (*tail*), lalu menyambungkan pointer *next* pada node terakhir tersebut menuju node baru. Validasi *try-except* diterapkan pada input harga dan stok untuk mencegah program berhenti (*crash*) apabila admin memasukkan nilai yang bukan angka. Setelah berhasil ditambahkan, data langsung disimpan secara permanen ke `data/menu.txt` melalui method `simpan_data_ke_txt()`.

### 3.3 Fitur Read



```
main.py - Tampilkan Menu - 80x32

DAFTAR MENU SARIAWAM

-- MAKANAN UTAMA --      -- SOUP & SALAD --      -- DIMSUM & SIOMAY --      -- ANEKA PASTA --
--
Nasi Ayam Bakar          Sup Krim Jagung          Siomay Ayam          Spaghetti Bolognese
Rp20.000                 Rp15.000                 Rp15.000             Rp20.000
Nasi Ayam Goreng         Sup Krim Jamur          Siomay Udang         Spaghetti Carbonara
Rp18.000                 Rp15.000                 Rp18.000             Rp22.000
Nasi Goreng Spesial      Salad Sayur             Dimsum Ayam          Macaroni Keju
Rp22.000                 Rp12.000                 Rp15.000             Rp18.000
Ayam Geprek              Salad Ayam              Dimsum Udang         Lasagna Sapi
Rp16.000                 Rp18.000                 Rp18.000             Rp25.000

... (kategori Cemilan, Mie, Kopi, Non-Kopi ditampilkan pada baris berikutnya)

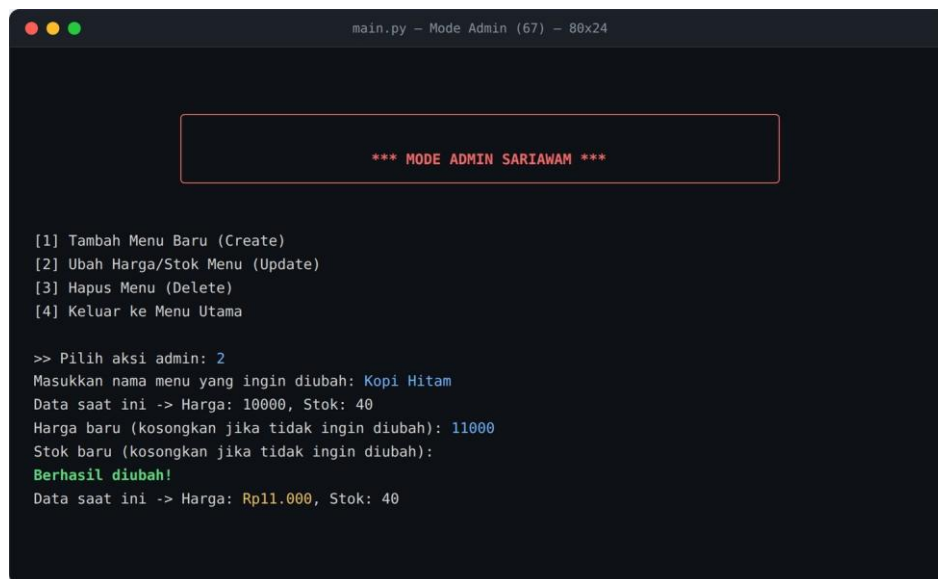
-- Opsi Menu --
[1] Cari Hidangan (Ketik Nama)
[2] Urutkan Harga Termahal - Termurah
[3] Urutkan Harga Termurah - Termahal
[4] Kembali ke Layanan Utama

>> Pilih opsi: 1
```

Gambar 3.3 Fitur Read — Tampilan daftar menu

Fitur Read diimplementasikan pada method `tampilkan_menu()`, yang melakukan traversal terhadap seluruh node pada *Single Linked List*. Mengelompokkan menu berdasarkan jenis atau kategori ke dalam sebuah *dictionary*, kemudian menampilkannya dalam tabel empat kolom menggunakan komponen *Table* dan *Panel* dari pustaka *rich*. Setiap kategori menu ditampilkan lengkap dengan nama menu dan harga yang telah di ubah ke dalam format mata uang Rupiah.

### 3.4 Fitur Update



```
main.py - Mode Admin (67) - 80x24

*** MODE ADMIN SARIWAM ***

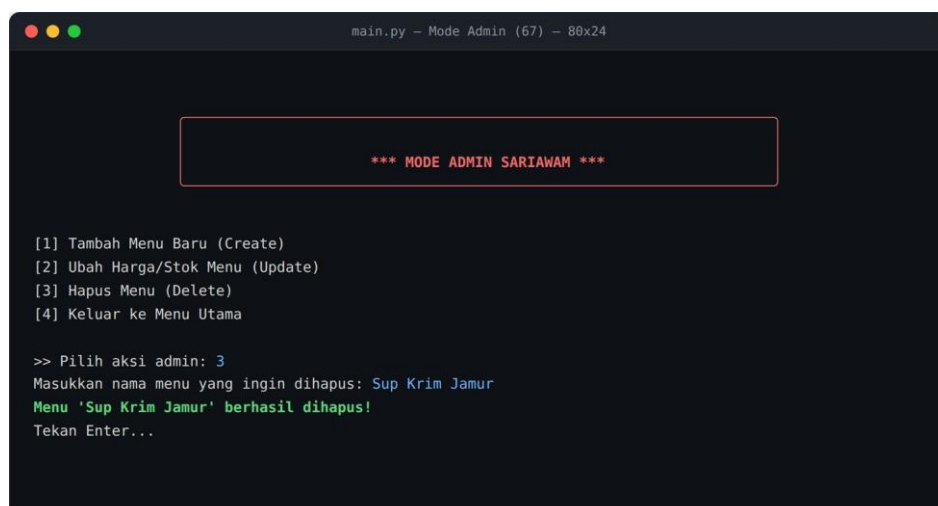
[1] Tambah Menu Baru (Create)
[2] Ubah Harga/Stok Menu (Update)
[3] Hapus Menu (Delete)
[4] Keluar ke Menu Utama

>> Pilih aksi admin: 2
Masukkan nama menu yang ingin diubah: Kopi Hitam
Data saat ini -> Harga: 10000, Stok: 40
Harga baru (kosongkan jika tidak ingin diubah): 11000
Stok baru (kosongkan jika tidak ingin diubah):
Berhasil diubah!
Data saat ini -> Harga: Rp11.000, Stok: 40
```

Gambar 3.4 Fitur Update — Ubah harga/stok menu

Fitur Update memungkinkan admin mengubah harga dan/atau stok menu yang sudah ada. Sistem terlebih dahulu mencari node menu yang dimaksud menggunakan method `cari_menu()`, kemudian menampilkan data harga dan stok saat ini sebagai referensi. Admin dapat memasukkan nilai baru atau mengosongkan input apabila tidak ingin mengubah salah satu atribut. Karena objek Node pada Python bersifat *mutable*, pengubahan nilai cukup dilakukan langsung terhadap atribut node (`node.harga`, `node.stok`) tanpa perlu membuat node baru, kemudian perubahan tersebut disimpan kembali ke `data/menu.txt`.

### 3.5 Fitur Delete



```
main.py - Mode Admin (67) - 80x24

*** MODE ADMIN SARIWAM ***

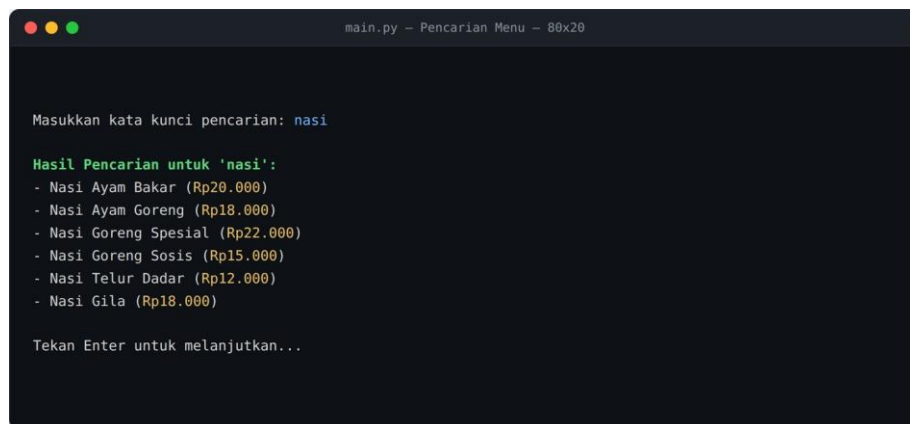
[1] Tambah Menu Baru (Create)
[2] Ubah Harga/Stok Menu (Update)
[3] Hapus Menu (Delete)
[4] Keluar ke Menu Utama

>> Pilih aksi admin: 3
Masukkan nama menu yang ingin dihapus: Sup Krim Jamur
Menu 'Sup Krim Jamur' berhasil dihapus!
Tekan Enter...
```

Gambar 3.5 Fitur Delete — Hapus menu

Fitur penghapusan menu dilakukan melalui penelusuran manual pada `main.py` dengan mempertahankan referensi node sebelumnya (*previous*) dan node saat ini (*current*). Ketika nama menu yang dicari ditemukan, pointer *next* pada node *prev* akan langsung diarahkan ke `current.next`, sehingga node target terlepas dari rantai *Single Linked List* tanpa perlu menghapus elemen secara fisik satu per satu. Penanganan khusus turut disediakan untuk kasus penghapusan node head, yaitu dengan mengarahkan head langsung ke `current.next`.

### 3.6 Fitur Searching



```
main.py - Pencarian Menu - 80x20

Masukkan kata kunci pencarian: nasi

Hasil Pencarian untuk 'nasi':
- Nasi Ayam Bakar (Rp20.000)
- Nasi Ayam Goreng (Rp18.000)
- Nasi Goreng Spesial (Rp22.000)
- Nasi Goreng Sosis (Rp15.000)
- Nasi Telur Dadar (Rp12.000)
- Nasi Gila (Rp18.000)

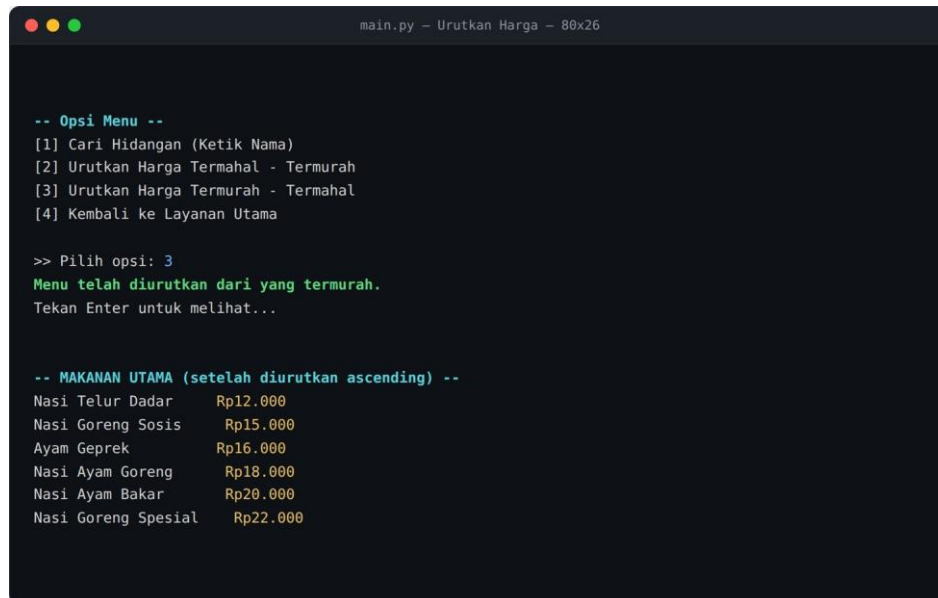
Tekan Enter untuk melanjutkan...
```

Gambar 3.6 Fitur Searching — Pencarian menu berdasarkan kata kunci

Fitur pencarian pada layanan utama menggunakan method `cari_menu_sebagian()`, yang menerapkan algoritma *Sequential Search* dengan pendekatan *partial match* sehingga pengguna tidak perlu mengetikkan nama menu secara lengkap. Sebagai contoh pada Gambar 3.6, kata kunci ‘nasi’ berhasil menampilkan seluruh menu yang mengandung kata tersebut, seperti Nasi Ayam Bakar, Nasi Goreng Spesial, dan Nasi Telur Dadar, lengkap dengan harga masing-masing.



### 3.7 Fitur Sorting



```
-- Opsi Menu --
[1] Cari Hidangan (Ketik Nama)
[2] Urutkan Harga Termahal - Termurah
[3] Urutkan Harga Termurah - Termahal
[4] Kembali ke Layanan Utama

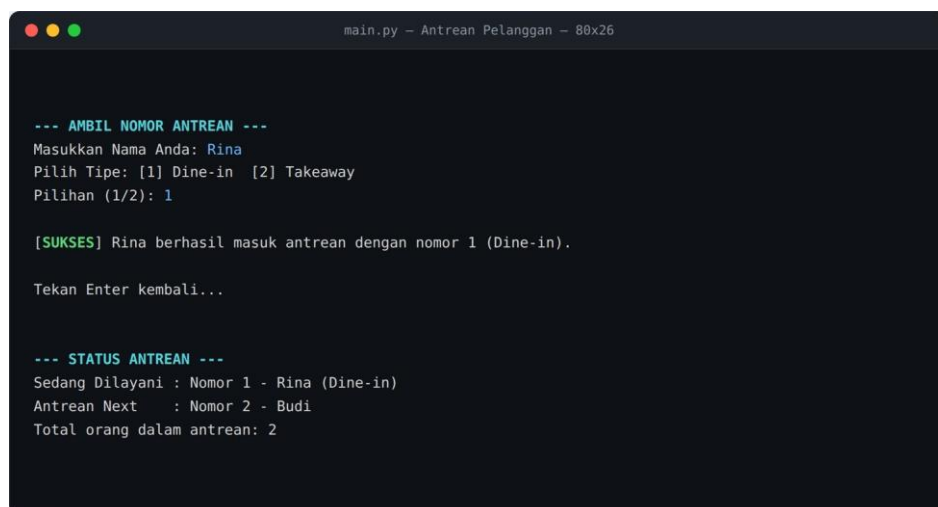
>> Pilih opsi: 3
Menu telah diurutkan dari yang termurah.
Tekan Enter untuk melihat...

-- MAKANAN UTAMA (setelah diurutkan ascending) --
Nasi Telur Dadar      Rp12.000
Nasi Goreng Sosis    Rp15.000
Ayam Geprek          Rp16.000
Nasi Ayam Goreng     Rp18.000
Nasi Ayam Bakar      Rp20.000
Nasi Goreng Spesial  Rp22.000
```

Gambar 3.7 Fitur Sorting — Pengurutan harga menu

Fitur pengurutan diimplementasikan pada method `urutkan_harga(ascending)`, yang menerapkan algoritma *Bubble Sort*. Pengguna dapat memilih urutan termurah ke termahal (`ascending=True`) atau termahal ke termurah (`ascending=False`). Pada Gambar 3.7, hasil pengurutan ascending menunjukkan menu kategori Makanan Utama yang telah terurut dari harga terendah (Nasi Telur Dadar, Rp12.000) hingga harga tertinggi (Nasi Goreng Spesial, Rp22.000).

### 3.8 Fitur Antrean Pelanggan (Queue)



```
--- AMBIL NOMOR ANTREAN ---
Masukkan Nama Anda: Rina
Pilih Tipe: [1] Dine-in [2] Takeaway
Pilihan (1/2): 1

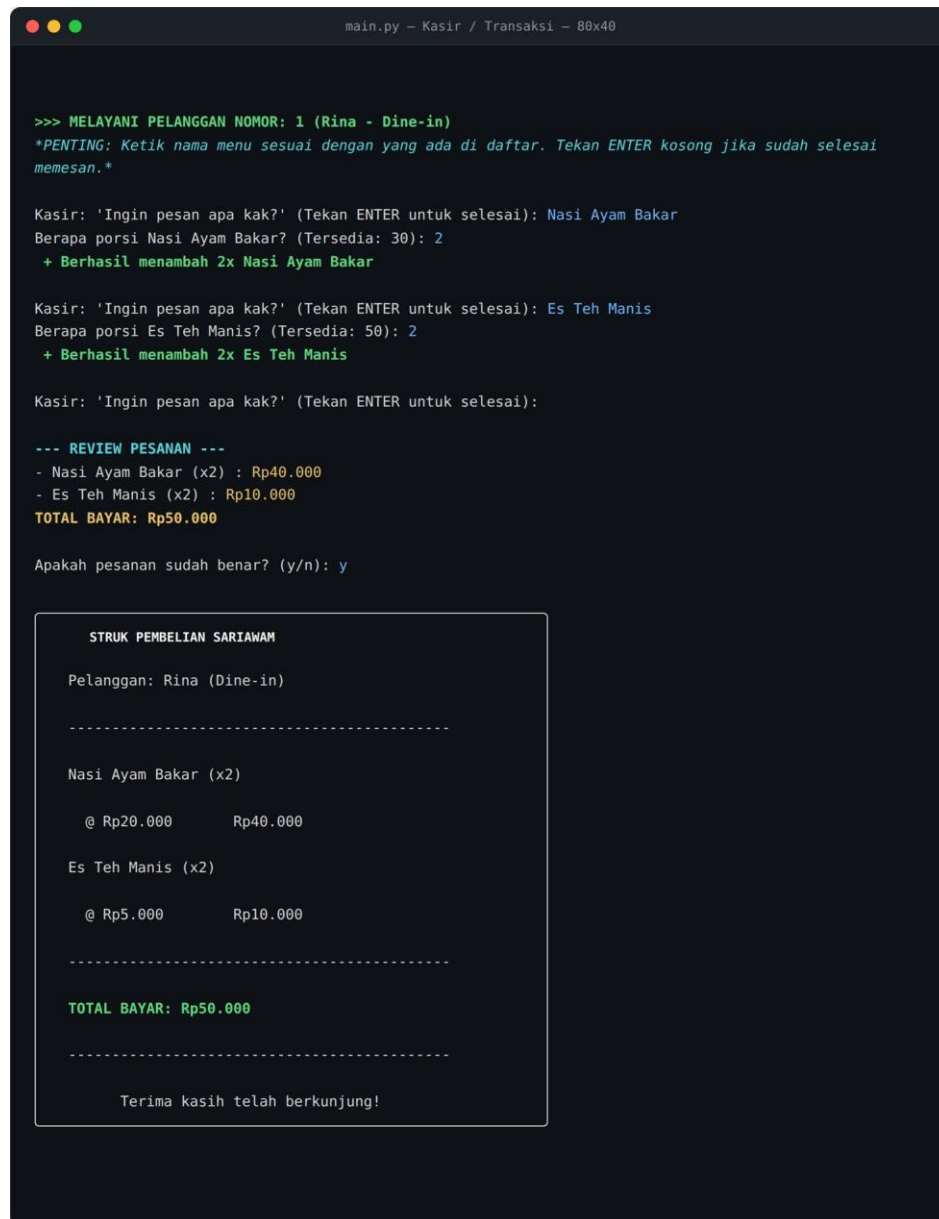
[SUKSES] Rina berhasil masuk antrean dengan nomor 1 (Dine-in).
Tekan Enter kembali...

--- STATUS ANTREAN ---
Sedang Dilayani : Nomor 1 - Rina (Dine-in)
Antrean Next    : Nomor 2 - Budi
Total orang dalam antrean: 2
```

*Gambar 3.8 Fitur antrean pelanggan — ambil nomor dan status antrean*

Fitur antrean diimplementasikan pada class `QueuePesanan` dengan struktur data *Queue* berbasis list bawaan Python. Operasi *enqueue* dilakukan melalui method `ambil_antrean()`, yang menambahkan data pelanggan (nomor antrean otomatis bertambah, nama, dan tipe pesanan *Dine-in* atau *Takeaway*) ke akhir antrean menggunakan `append()`. Status antrean dapat dipantau melalui method `tampilkan_antrean()`, yang menampilkan pelanggan yang sedang dilayani (elemen terdepan/indeks ke-0), pelanggan berikutnya (indeks ke-1), serta total pelanggan yang sedang mengantre.

### 3.9 Fitur Transaksi Kasir



```
main.py - Kasir / Transaksi - 80x40

>>> MELAYANI PELANGGAN NOMOR: 1 (Rina - Dine-in)
*PENTING: Ketik nama menu sesuai dengan yang ada di daftar. Tekan ENTER kosong jika sudah selesai
memesan.*

Kasir: 'Ingin pesan apa kak?' (Tekan ENTER untuk selesai): Nasi Ayam Bakar
Berapa porsi Nasi Ayam Bakar? (Tersedia: 30): 2
+ Berhasil menambah 2x Nasi Ayam Bakar

Kasir: 'Ingin pesan apa kak?' (Tekan ENTER untuk selesai): Es Teh Manis
Berapa porsi Es Teh Manis? (Tersedia: 50): 2
+ Berhasil menambah 2x Es Teh Manis

Kasir: 'Ingin pesan apa kak?' (Tekan ENTER untuk selesai):

--- REVIEW PESANAN ---
- Nasi Ayam Bakar (x2) : Rp40.000
- Es Teh Manis (x2) : Rp10.000
TOTAL BAYAR: Rp50.000

Apakah pesanan sudah benar? (y/n): y

STRUK PEMBELIAN SARIAWAM
Pelanggan: Rina (Dine-in)
-----
Nasi Ayam Bakar (x2)
    @ Rp20.000      Rp40.000
Es Teh Manis (x2)
    @ Rp5.000       Rp10.000
-----
TOTAL BAYAR: Rp50.000
-----
Terima kasih telah berkunjung!
```

Gambar 3.9 Fitur transaksi kasir dan struk pembelian

Proses transaksi kasir diimplementasikan pada method `layani_pelanggan()`, yang memanggil pelanggan terdepan dalam antrian (indeks ke-0) tanpa langsung menghapusnya (*dequeue*) hingga transaksi selesai dikonfirmasi. Kasir dapat memasukkan nama menu yang dipesan beserta jumlahnya secara berulang hingga menekan ENTER kosong untuk mengakhiri pemesanan. Sistem secara otomatis menghitung subtotal dan total belanja, serta menggabungkan jumlah pesanan apabila menu yang sama dipesan lebih dari satu kali. Setelah pelanggan mengonfirmasi pesanan (input 'y'), stok menu dikurangi melalui method

kurangi\_stok(), pelanggan dikeluarkan dari antrean melalui pop(0) (operasi dequeue), dan struk pembelian dicetak melalui method \_cetak\_struk() yang menampilkan rincian pesanan dan total pembayaran dalam sebuah Panel.

### 3.10 Penyimpanan Data (File Handling)

Data menu disimpan secara permanen pada berkas teks data/menu.txt dengan format menyerupai CSV, yaitu empat kolom yang dipisahkan oleh tanda koma (jenis,nama,harga,stok), dengan satu baris untuk setiap item menu. Berikut potongan isi berkas data/menu.txt:

#### **data/menu.txt (cuplikan)**

```
Makanan Utama,Nasi Ayam Bakar,20000,30 Makanan  
Utama,Nasi Ayam Goreng,18000,45 Makanan Utama,Nasi Goreng  
Spesial,22000,25 Makanan Utama,Nasi Goreng Sosis,15000,40  
Makanan Utama,Nasi Telur Dadar,12000,50 Makanan Utama,Nasi  
Gila,18000,20  
Makanan Utama,Ayam Geprek,16000,40 Makanan  
Utama,Pecel Lele,15000,25
```

Proses pembacaan berkas dilakukan oleh method muat\_dari\_txt() pada class LinkedListMenu, yang membaca berkas baris demi baris, memisahkan (*split*) setiap baris berdasarkan koma, kemudian menambahkan hasil pemecahan tersebut ke dalam *Single Linked List* melalui tambah\_di\_akhir(). Apabila berkas tidak ditemukan (*FileNotFoundException*), program akan menampilkan pesan kesalahan tanpa menghentikan eksekusi program secara paksa.

Sebaliknya, proses penulisan dilakukan oleh method simpan\_data\_ke\_txt() pada class QueuePesanan, yang mengambil seluruh data terbaru dari Single Linked List melalui method ambil\_semua\_data(), kemudian menuliskan ulang seluruh isi berkas menggunakan mode 'w'. Method ini dipanggil otomatis setiap kali terjadi perubahan data, baik melalui mode admin (*Create, Update, Delete*) maupun melalui transaksi kasir yang mengurangi stok.

### 3.11 Validasi Input dan Error Handling

Untuk menjaga kestabilan program, berbagai mekanisme validasi input dan penanganan kesalahan (*error handling*) diterapkan pada seluruh bagian aplikasi, sebagaimana dirangkum pada Tabel 3.1 berikut.

| Kesalahan Input                                                                      | Penanganan                                                                                                                                           |
|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Karakter non-angka dimasukkan pada input harga, stok, atau jumlah pesanan            | Program menangkap ValueError melalui blok try-except dan meminta pengguna memasukkan ulang nilai yang valid berupa angka tanpa menghentikan program. |
| Nama menu yang dicari, diubah, atau dihapus tidak ditemukan dalam database           | Program menampilkan pesan “Menu tidak ditemukan” dan dikembalikan ke menu sebelumnya.                                                                |
| Folder data atau berkas menu.txt belum tersedia saat program pertama kali dijalankan | Program otomatis membuat folder data/ menggunakan os.makedirs() apabila belum ada, sehingga program tidak mengalami crash.                           |
| Jumlah pesanan melebihi stok yang tersedia, atau bernilai nol/negatif                | Program menampilkan pesan kegagalan dan meminta kasir memasukkan jumlah pesanan ulang.                                                               |
| Input pilihan menu/tipe antrean di luar opsi yang tersedia (selain 1/2 atau y/n)     | Program menampilkan pesan “Input tidak valid” dan mengulang permintaan input hingga pengguna memasukkan nilai yang sesuai.                           |

## BAB IV

### PENGUJIAN DAN ANALISIS

#### 4.1 Skenario Pengujian

Pengujian dilakukan secara fungsional terhadap seluruh fitur utama aplikasi untuk memastikan setiap fitur berjalan sesuai dengan rancangan. Tabel 4.1 merangkum skenario pengujian beserta hasilnya.

| No | Skenario                                                          | Hasil                                                                   |
|----|-------------------------------------------------------------------|-------------------------------------------------------------------------|
| 1  | Menambah data menu baru melalui mode admin ( <i>Create</i> )      | Berhasil, data tersimpan di data/menu.txt                               |
| 2  | Mengubah harga dan stok menu melalui mode admin ( <i>Update</i> ) | Berhasil, perubahan tersimpan dan tampil sesuai input baru              |
| 3  | Menghapus menu melalui mode admin ( <i>Delete</i> )               | Berhasil, node terhapus <i>dari Single Linked List</i> dan berkas txt   |
| 4  | Mengurutkan harga menu ( <i>ascending</i> dan <i>descending</i> ) | Berhasil, urutan harga sesuai pilihan pengguna                          |
| 5  | Mencari menu berdasarkan kata kunci sebagian                      | Berhasil, seluruh menu yang sesuai kata kunci ditampilkan               |
| 6  | Mengambil nomor antrean ( <i>Dine-in</i> dan <i>Takeaway</i> )    | Berhasil, nomor antrean bertambah otomatis                              |
| 7  | Transaksi kasir dengan stok mencukupi                             | Berhasil, stok berkurang, struk tercetak, pelanggan keluar dari antrean |
| 8  | Transaksi kasir dengan jumlah pesanan melebihi stok               | Berhasil ditangani, sistem menolak dan meminta input ulang              |
| 9  | Input harga/stok berupa huruf (non-numerik)                       | Berhasil ditangani melalui try-except, program tidak <i>crash</i>       |
| 10 | Program dijalankan tanpa folder data/ atau berkas menu.txt        | Berhasil, folder otomatis dibuat dan pesan informatif ditampilkan       |

## **4.2 Kelebihan Sistem**

1. Data menu tersimpan secara permanen pada berkas teks dan tetap tersedia meskipun program dihentikan dan dijalankan kembali.
2. Program dilengkapi validasi input yang memadai sehingga tidak mudah mengalami crash akibat kesalahan input pengguna.
3. Antarmuka CLI yang interaktif dan berwarna (menggunakan pustaka rich) membuat aplikasi lebih nyaman digunakan dibandingkan CLI konvensional.
4. Mekanisme antrean FIFO menjamin keadilan dalam urutan pelayanan pelanggan.
5. Mode admin tersembunyi memisahkan akses pengelolaan data dari akses pengguna umum/pelanggan.

## **4.3 Keterbatasan Sistem**

1. Aplikasi masih berbasis CLI sehingga kurang ramah bagi pengguna awam yang terbiasa dengan antarmuka grafis (GUI).
2. Belum mendukung sistem basis data (database) sehingga kurang optimal untuk skala data yang sangat besar.
3. Belum mendukung multiuser atau multi-kasir secara bersamaan, karena akses program bersifat satu sesi terminal.
4. Mode admin hanya dilindungi oleh kode akses sederhana ('67') tanpa sistem autentikasi (login dan password) yang lebih aman.
5. Algoritma Bubble Sort yang digunakan kurang efisien apabila diterapkan pada jumlah data menu yang sangat besar.

## BAB V

### KENDALA DAN SOLUSI

| Kendala                                                                                                                                           | Solusi                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Data menu dan antrian berisiko hilang apabila program dihentikan secara paksa                                                                     | Menerapkan penyimpanan otomatis (auto-save) ke data/menu.txt setiap kali terjadi perubahan data                        |
| Program berisiko berhenti (crash) ketika pengguna memasukkan data yang tidak sesuai tipe, misalnya huruf pada input harga                         | Menambahkan blok try-except pada seluruh input numerik penting                                                         |
| Penukaran data antar-node pada <i>Single Linked List</i> saat proses sorting berisiko merusak rantai pointer apabila dilakukan secara sembarangan | Menukar isi atau <i>value</i> antar-node yang bersebelahan, bukan menukar pointer <i>next</i> -nya                     |
| Stok menu yang sama berisiko tercatat ganda pada keranjang pesanan apabila pelanggan memesan menu yang sama lebih dari satu kali                  | Menambahkan pengecekan pada keranjang pesanan untuk menggabungkan (menjumlahkan) pesanan dengan nama menu yang identik |

Selama proses pengembangan, tantangan utama yang dihadapi kelompok adalah memastikan konsistensi data antara struktur *Single Linked List* di memori dengan berkas data/menu.txt, terutama ketika perubahan data terjadi dari dua sisi berbeda, yaitu mode admin dan proses transaksi kasir. Kelompok perlu memastikan method `simpan_data_ke_txt()` selalu dipanggil setelah perubahan data agar tidak terjadi perbedaan yang signifikan antara data di memori dan data pada berkas.

Tantangan lain yang dihadapi adalah mengintegrasikan dua struktur data yang berbeda, yaitu *Single Linked List* dan *Queue* ke dalam satu alur program yang utuh pada main.py. Tantangan ini diselesaikan dengan membagi tanggung jawab antar anggota kelompok sesuai modul masing-masing kemudian melakukan pengujian integrasi secara bersama-sama sebelum laporan akhir ini disusun.



## BAB VI

### PEMBAGIAN TUGAS ANGGOTA

| Nama               | Tugas                                                                                                                                                                                      |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Fardhan Al Biansya | Perancangan dan implementasi struktur data menu ( <i>Single Linked List</i> ), termasuk fitur <i>Create</i> , <i>Read</i> , <i>Searching</i> , dan <i>Sorting</i> .                        |
| Hirzi Haidi Haqqi  | Perancangan dan implementasi struktur data antrean ( <i>Queue</i> ), proses transaksi kasir, serta seluruh mekanisme file handling (membaca dan menyimpan data ke <code>menu.txt</code> ). |
| Dawam Raihan Zufar | Manajer proyek ( <i>project manager</i> ), perancangan antarmuka CLI/UX menggunakan pustaka rich, serta implementasi mode admin ( <i>Update</i> dan <i>Delete</i> ).                       |

Pembagian tugas di atas merupakan fokus utama masing-masing anggota tetapi, seluruh anggota kelompok turut terlibat dalam diskusi perancangan, debugging, serta pengujian aplikasi secara menyeluruh. Kontribusi masing-masing anggota turut tercermin pada riwayat commit (*commit history*) repository GitHub kelompok sebagaimana tercantum pada Lampiran D.

## BAB VII

### KESIMPULAN DAN SARAN

#### 7.1 Kesimpulan

Berdasarkan hasil pengerjaan dan pembahasan proyek ini, maka dapat disimpulkan bahwa:

1. Aplikasi SARIAWAM berhasil dikembangkan sebagai sistem kasir restoran berbasis CLI yang menerapkan struktur data *Single Linked List* untuk pengelolaan menu dan *Queue* (FIFO) untuk pengelolaan antrean pelanggan.
2. Seluruh fitur utama yang direncanakan, meliputi CRUD data menu, pencarian, pengurutan, manajemen antrean, transaksi kasir, dan penyimpanan data permanen, telah berhasil diimplementasikan dan diuji dengan hasil yang sesuai harapan.
3. Penerapan struktur data dan algoritma yang dipelajari pada mata kuliah Algoritma dan Struktur Data terbukti dapat diterapkan untuk menyelesaikan permasalahan nyata dalam pengelolaan operasional restoran skala kecil-menengah.

#### 7.2 Saran Pengembangan

1. Mengganti media penyimpanan dari berkas teks (.txt) menjadi sistem basis data (misalnya SQLite atau MySQL) agar lebih andal untuk skala data yang lebih besar.
2. Mengembangkan antarmuka grafis (GUI) atau aplikasi berbasis web agar lebih mudah digunakan oleh pengguna awam.
3. Menambahkan sistem autentikasi (login dan password) yang lebih aman untuk mode admin, menggantikan kode akses sederhana yang digunakan saat ini.
4. Mengganti algoritma *Bubble Sort* dengan algoritma sorting yang lebih efisien, seperti *Merge Sort*, apabila jumlah data menu bertambah signifikan di masa mendatang.

## DAFTAR PUSTAKA

- Simbolon, A. P. H., Sagala, K. F., Tanjung, M. R. A., Zai, T. S. W., & Ramadhani, F. (2024). Implementasi Sistem Pengelolaan Pesanan Menu Restoran Berbasis Stack dan Queue. *bit-Tech*, 7(2), 484-493.
- Banjarnahor, J. (2022). Pemanfaatan Link List Untuk Mengatasi database Tidak Normal. *LOFIAN: Jurnal Teknologi Informasi dan Komunikasi*, 2(1), 16-23.
- Septiani, C. S., Sihombing, T. R. M. P., Valentine, V., Priskila, R., & Pranatawijaya, V. H. (2024). Pemanfaatan Kecerdasan Buatan Pada Implementasi Sequential Search Pencarian Hotel Di Kota Palangka Raya Berdasarkan Harga. *JATI (Jurnal Mahasiswa Teknik Informatika)*, 8(3), 3511-3517.
- Diatmika, N. G. R. S., Suarjaya, I. M. A. D., & Wibawa, K. S. (2022). Perancangan User Interface dan User Experience Aplikasi Pemesanan Menu Restoran. *Jurnal Ilmiah Teknologi dan Komputer*, 3(2), 1051-1060.

## LAMPIRAN

### Lampiran A. Link GitHub

Repositori kode sumber lengkap proyek ini dapat diakses melalui tautan berikut: <https://github.com/dwmrhnz/Kelompok3>

Repositori tersebut memuat seluruh source code (main.py, modules/linkedlist\_menu.py, modules/queue\_antrean.py), berkas data/menu.txt, serta README.md yang berisi dokumentasi proyek secara lengkap.

### Lampiran B. Screenshot Program

Seluruh tangkapan layar (screenshot) yang menunjukkan jalannya aplikasi SARIAWAM, mulai dari tampilan menu utama, fitur *Create, Read, Update, Delete, Searching, Sorting*, antrean pelanggan, hingga transaksi kasir dan struk pembelian, telah disertakan secara lengkap pada Bab III Implementasi Program (Gambar 3.1 sampai dengan Gambar 3.9).

### Lampiran C. Struktur Folder Proyek

#### Struktur direktori repositori Kelompok3

|                            |                                |
|----------------------------|--------------------------------|
| Kelompok3/                 |                                |
| ├── data/                  |                                |
| │   └── menu.txt           | # Database menu (format txt)   |
| ├── modules/               |                                |
| │   ├── linkedlist_menu.py | # Struktur data Linked List    |
| │   └── queue_antrean.py   | # Struktur data Queue          |
| ├── .gitignore             | # Konfigurasi pengabaian Git   |
| ├── main.py                | # File eksekusi utama aplikasi |
| └── README.md              | # Dokumentasi proyek           |

### Lampiran D. Commit History GitHub

Riwayat commit (*commit history*) yang mencerminkan kontribusi setiap anggota kelompok selama proses pengembangan dapat dilihat secara lengkap pada repositori GitHub yang tercantum pada Lampiran A, tepatnya pada menu “Commits” atau “Insights > Contributors” di halaman repositori tersebut.

## Lampiran E. Source Code Utama

Source code lengkap aplikasi SARIAWAM (main.py, modules/linkedlist\_menu.py, dan modules/queue\_antrean.py) disertakan pada halaman-halaman berikut.

### main.py

```
import os
from rich.console import Console
from rich.panel import Panel
from rich.align import Align
from modules.linkedlist_menu import LinkedListMenu
from modules.queue_antrean import QueuePesanan

console = Console(highlight=False)

def bersihkan_layar():
    """ Fungsi helper buat clear screen terminal. """
    os.system('cls' if os.name == 'nt' else 'clear')

def main():
    # inialisasi object list & queue daftar_menu =
    LinkedListMenu()
    antrean_kasir = QueuePesanan()

    # error handling kalo folder data belum ada if not
    os.path.exists('data'):
        os.makedirs('data')

    # load data menu awal dari txt ke memori (linked list) lokasi_file_txt = "data/menu.txt"
    daftar_menu.muat_dari_txt(lokalasi_file_txt)

    while True: # main loop aplikasi bersihkan_layar()

        # render UI panel judul pake library rich judul = Panel(
            Align.center("[bold yellow]SELAMAT DATANG DI RESTORAN SARIAWAM[/bold yellow]\n[italic
white]Fardhan - Hirzi - Dawam[/italic white]"),
            style="bold cyan", width=65
        )
        console.print(judul)

        console.print("\n[bold cyan]-- Silakan Pilih Layanan --[/bold cyan]")
        console.print("[1] Tampilkan Menu & Cari")
        console.print("[2] Ambil Nomor Antrean")
        console.print("[3] Tampilkan Antrean Saat Ini")
        console.print("[4] Menuju Kasir (Melayani Pelanggan)")
        console.print("[5] Keluar\n")

        pilihan = input(">> Masukkan pilihan Anda: ").strip()

        if pilihan == '1':
            while True: # nested loop buat sub-menu menu bersihkan_layar()
                daftar_menu.tampilkan_menu()

                console.print("\n[bold cyan]-- Opsi Menu --[/bold cyan]")
                console.print("[1] Cari Hidangan (Ketik Nama)")
                console.print("[2] Urutkan Harga Termahal - Termurah")
                console.print("[3] Urutkan Harga Termurah - Termahal")
                console.print("[4] Kembali ke Layanan Utama")

                opsi_menu = input(">> Pilih opsi: ").strip()
```

```

        if opsi_menu == '1':
            kata = input("Masukkan kata kunci pencarian: ").strip()
            hasil = daftar_menu.cari_menu_sebagian(kata)
            console.print(f"\n[bold green]Hasil Pencarian untuk '{kata}':

[/bold green]")

            if hasil:
                for h in hasil:
                    # format value integer ke string IDR
                    harga_format = f"Rp {h.harga:,}".replace(",", ".")
                    console.print(f"-
[/yellow]{harga_format}[/yellow]")
                else:
                    console.print("[red]Menu tidak ditemukan.[/red]")
                    input("\nTekan Enter untuk
melanjutkan...")

            elif opsi_menu == '2':
                daftar_menu.urutkan_harga(ascending=False) # sort desc
                console.print("[green]Menu telah diurutkan dari yang termahal.
input('Tekan Enter untuk
melihat...')

            elif opsi_menu == '3':
                daftar_menu.urutkan_harga(ascending=True) # sort asc
                console.print("[green]Menu telah diurutkan dari yang termurah.
input('Tekan Enter untuk
melihat...')

            elif opsi_menu == '4':
                break # exit sub-menu

    elif pilihan == '2':
        bersihkan_layar()
        console.print("[bold cyan]--- AMBIL NOMOR ANTREAN ---[/bold cyan]")
        nama = input("Masukkan Nama Anda: ").strip()
        if nama:
            console.print("Pilih Tipe: [1] Dine-in [2] Takeaway")
            while True:
                tipe_input = input("Pilihan (1/2): ").strip()
                if tipe_input == '1':
                    tipe = "Dine-in"
                    break
                elif tipe_input == '2':
                    tipe = "Takeaway"
                    break
            else:
                console.print("[bold red]Input tidak valid! Harap masukkan angka 1 atau 2.[/bold red]")
            antrean_kasir.ambil_antrean(nama, tipe)
            input("\nTekan Enter kembali...")

        elif pilihan == '3':
            bersihkan_layar()
            antrean_kasir.tampilkan_antrean()
            input("\nTekan Enter kembali...")

        elif pilihan == '4':
            bersihkan_layar()
            antrean_kasir.layani_pelanggan(daftar_menu) # hit function dequeue
            input("\nTekan Enter kembali...")

        elif pilihan == '5':
            bersihkan_layar()
            console.print("[bold green]Terima kasih telah berkunjung ke Sariawam.
Sampai jumpa![/bold green]")
            break

    # secret code buat masuk backdoor admin
    elif pilihan == '67':

```

```

while True: # nested loop buat menu admin (CRUD) bersihkan layar()
    admin_panel = Panel(Align.center("[bold red]*** MODE ADMIN SARIAWAM
***[/bold red]"), width=65)
    console.print(admin_panel)
    console.print("[1] Tambah Menu Baru (Create)") console.print("[2] Ubah
    Harga/Stok Menu (Update)") console.print("[3] Hapus Menu (Delete)")
    console.print("[4] Keluar ke Menu Utama\n")

    admin_pilihan = input(">>> Pilih aksi admin: ").strip()

    if admin_pilihan == '1': # CREATE
        jenis = input("Jenis (Makanan Utama/Kopi/dll): ") nama_menu = input("Nama
        Menu: ")
        try:
            # validasi input pake try-except biar ga crash kalo
            harga = int(input("Harga: ")) stok =
            int(input("Stok: "))
            daftar_menu.tambah_di_akhir(jenis, nama_menu, harga, stok)
            antrean_kasir.simpan_data_ke_txt(daftar_menu,

            console.print("[green]Berhasil ditambah![/green]") except ValueError:
            console.print("[red]Harga/Stok harus angka![/red]") input("Tekan Enter...")

        elif admin_pilihan == '2': # UPDATE
            nama_ubah = input("Masukkan nama menu yang ingin diubah:
            ").strip()

            node = daftar_menu.cari_menu(nama_ubah)
            if node: # ngecek kalo nodenya beneran ada console.print(f"Data saat ini -> Harga:
                {node.harga},

            try:
                hrg_baru = input("Harga baru (kosongkan jika tidak stk_baru = input("Stok baru
                (kosongkan jika tidak

                # update value node kalo input ga kosong if hrg_baru:
                node.harga = int(hrg_baru) if stk_baru: node.stok =
                int(stk_baru)

                antrean_kasir.simpan_data_ke_txt(daftar_menu,

                harga_skrng = f"Rp{node.harga:,}".replace(",", ".") console.print("[green]Berhasil
                diubah![/green]") console.print(f"Data saat ini -> Harga: [yellow]
                {harga_skrng}[yellow], Stok: {node.stok}")
                except ValueError:
                console.print("[red]Input harus angka![/red]")

            else:
                console.print("[red]Menu tidak ditemukan![/red]") input("Tekan Enter...")

        elif admin_pilihan == '3': # DELETE
            nama_hapus = input("Masukkan nama menu yang ingin dihapus:
            ").strip()

            # manual traversal buat delete node current =
            daftar_menu.head
            prev = None ditemukan =
            False

            while current:
                if current.nama.lower() == nama_hapus.lower(): ditemukan = True
                if prev: # target ada di tengah/belakang list prev.next = current.next # bypass /
                    unlink node
                else: # target itu head-nya sendiri

```

```

        daftar_menu.head = current.next break
    prev = current
    current = current.next

    if ditemukan:
        # write ulang ke txt kalo berhasil kehapus antrean_kasir.simpan_data_ke_txt(daftar_menu,
        lokasi_file_txt)
        console.print(f"[green]Menu '{nama_hapus}' berhasil
    dihapus![/green]")
    else:
        console.print(f"[red]Menu tidak ditemukan![/red]") input("Tekan Enter...")

    elif admin_pilihan == '4': break # exit
    backdoor
else:

    console.print(f"\n[red]Pilihan tidak valid.[/red]") input("Tekan Enter...")

if __name__ == "__main__": main()

```



## modules/linkedlist\_menu.py

```
from rich.console import Console from
rich.table import Table from rich.panel import
Panel from rich import box

class NodeMenu:
    """ Blueprint buat tiap node menu di Linked List. """ def __init__(self, jenis, nama,
    harga, stok):
        self.jenis = jenis self.nama = nama
        self.harga = int(harga) # casting ke int biar aman buat dihitung self.stok = int(stok)
        self.next = None # pointer ke node selanjutnya

class LinkedListMenu:
    """ Class utama buat operasi Single Linked List (CRUD & Sorting). """ def __init__(self):
        self.head = None # head awal kosong

    def tambah_di_akhir(self, jenis, nama, harga, stok):
        """ Bikin node baru terus ditaruh di paling ujung (tail). """ node_baru = NodeMenu(jenis, nama,
        harga, stok) # buat node baru

        # kalo list masih kosong, node baru langsung jadi head if not self.head:
            self.head = node_baru return

        # jika sudah ada, traverse sampe nemu ujungnya current = self.head
        while current.next: current =
            current.next
        # program berjalan sampai node terakhir

        # sambungin pointer node terakhir ke node baru current.next = node_baru

    def muat_dari_txt(self, nama_file="menu.txt"):
        """ Baca data dari file txt, terus masukin ke Linked List. """ try:
            with open(nama_file, 'r') as file:
                for line in file: # membaca tiap baris
                    line = line.strip() # bersihin enter/spasi berlebih if not line:
                        continue # skip baris kosong

                    # split string dari format txt
                    parts = line.split(',') # memisahkan data dengan koma if len(parts) == 4:
                        jenis, nama, harga, stok = parts self.tambah_di_akhir(jenis.strip(), nama.strip(), harga,
stok)
        except FileNotFoundError:
            print(f"[red][ERROR][red] File {nama_file} nggak ketemu!")

    def tampilkan_menu(self):
        """ Print UI menu pakai library rich (dibuat 4 kolom). """ console = Console(highlight=False)
        if not self.head:
            console.print("[bold red]Daftar menu masih kosong.[/bold red]") return

        # grouping menu ke dictionary biar gampang di-print per kategori kategori_menu = {} #
        mengelompokkan menu berdasarkan kategori current = self.head # traversal dari head sampe ujung
        list
        while current:
            if current.jenis not in kategori_menu:
```

```

        kategori_menu[current.jenis] = [] # list kosong untuk menyimpan
semua menu
        kategori_menu[current.jenis].append(current) # menambahkan node menu
ke kategori yang sesuai.
        current = current.next

# setup tabel layout simple menggunakan rich
table = Table(show_header=False, box=box.SIMPLE, expand=True) for _ in range(4):
    table.add_column(justify="left", ratio=1)

nama_kategori = list(kategori_menu.keys()) # mengambil nama kategori dari dictionary

# pecah kategori jadi per 4 kolom
for i in range(0, len(nama_kategori), 4): chunk =
    nama_kategori[i:i+4]

# membuat judul kategori
judul_cols = [f"[bold cyan]{k.upper()} --[/bold cyan]" for k in
chunk]

while len(judul_cols) < 4: judul_cols.append("")
table.add_row(*judul_cols) # menambah kolom kosong

isi_cols = []
for k in chunk: # memproses setiap kategori teks_menu = ""
    for item in kategori_menu[k]:

        # convert integer ke string format rupiah harga_format =
        f"Rp {item.harga:,}".replace(",", ".") #susun teks menu
        teks_menu += f"[white]{item.nama}[/white]\n[yellow]
{harga_format}[/yellow]\n\n"
        isi_cols.append(teks_menu.strip())

while len(isi_cols) < 4: isi_cols.append("")
table.add_row(*isi_cols) table.add_row("", "", "", "")

panel = Panel(table, title="[bold magenta] DAFTAR MENU SARIAWAM [/bold magenta]",
border_style="magenta")
console.print(panel)

def cari_menu(self, nama_menu):
    """ Cari menu by nama exact. Return object Node kalo ketemu. """ current = self.head
    while current: # traversal dari head
        # di-lower() semua biar case-insensitive
        if current.nama.lower() == nama_menu.lower(): # membandingkan nama
menu
            return current
        current = current.next
    return None #jika cocok

def cari_menu_sebagian(self, keyword):
    """ Fitur search pakai keyword (partial match). Return list of Nodes. """ hasil = []
    current = self.head
    while current:
        if keyword.lower() in current.nama.lower(): # cek setiap node hasil.append(current) # data yang cocok
            masuk list hasil
        current = current.next
    return hasil

def urutkan_harga(self, ascending=True):
    """ Sorting Linked List pakai Bubble Sort (cuma swap value/datanya). """ if not self.head or not
self.head.next:
    return # skip kalo list kosong atau cuma 1 node

```

```

swapped = True # apakah masih ada pertukaran while swapped:
    swapped = False
    current = self.head # mulai pemeriksaan dari node pertama while current.next:
        # cek kondisi asc / desc if ascending:
        kondisi_tukar = current.harga > current.next.harga else:
            kondisi_tukar = current.harga < current.next.harga

        # eksekusi swap data kalo kondisi terpenuhi if kondisi_tukar:
            # swap jenis
            current.jenis, current.next.jenis = current.next.jenis,
current.jenis

            # swap nama
            current.nama, current.next.nama = current.next.nama,
current.nama

            # swap harga
            current.harga, current.next.harga = current.next.harga,
current.harga

            # swap stok
            current.stok, current.next.stok = current.next.stok,
current.stok

        swapped = True

    current = current.next # lanjut periksa node selanjutnya

def kurangi_stok(self, nama_menu, jumlah):
    """ Update stok node waktu ada pesanan di kasir. """ node =
    self.cari_menu(nama_menu)
    if node:
        if node.stok >= jumlah: # cek node.stok -= jumlah #
            kurangi return True
        else:
            print(f"[Peringatan] Stok {node.nama} kurang! Sisa: {node.stok}") return False
    else:
        print(f"[Peringatan] Menu '{nama_menu}' ga ketemu di database.") return False

def ambil_semua_data(self):
    """ Ambil semua data list buat di-rewrite balik ke txt. """ list_data = [] # buat list kosong
    current = self.head
    while current: # traversal seluruh linkedlist
        # format ulang stringnya persis kayak format txt awal baris_data =
        f"{current.jenis},{current.nama},{current.harga},
{current.stok}"
        list_data.append(baris_data) current =
        current.next

    return list_data # kembalikan seluruh data dalam bentuk list.

```

## modules/queue\_antrean.py

```
import os
from rich.console import Console
from rich.panel import Panel

class QueuePesanan:
    """ Class buat urus antrean FIFO (First In First Out) pakai list Python. """

    def __init__(self):
        self.antrian = [] # list buat nampung antrean
        self.nomor_terakhir = 0 # nyimpen nomor terakhir biar auto-increment

    def ambil_antrean(self, nama_pelanggan, tipe_pesanan): """ Enqueue: masukan
        pelanggan baru ke antrean. """ console = Console(highlight=False)
        self.nomor_terakhir += 1
        data_pelanggan = {
            "nomor": self.nomor_terakhir, "nama":
            nama_pelanggan, "tipe": tipe_pesanan
        }
        self.antrian.append(data_pelanggan) console.print(f"\n[[green]SUKSES[/green]] {nama_pelanggan} berhasil
        masuk
        antrean dengan nomor {self.nomor_terakhir} ({tipe_pesanan}).")

    def tampilkan_antrean(self):
        """ Intip isi antrean (siapa yang lagi dilayani, next siapa, total antrean). """
        console = Console(highlight=False)
        console.print("[bold cyan]--- STATUS ANTREAN ---[/bold cyan]") total_antrean =
        len(self.antrian)

        if total_antrean == 0:
            print("Antrean saat ini kosong.") return

        # ambil data index 0 (paling depan) sedang_dilayani =
        self.antrian[0]
        print(f"Sedang Dilayani : Nomor {sedang_dilayani['nomor']} -
        {sedang_dilayani['nama']} ({sedang_dilayani['tipe']})")

        # cek kalo masih ada antrean di belakangnya (index 1) if total_antrean > 1:
        berikutnya = self.antrian[1]
        print(f"Antrean Next : Nomor {berikutnya['nomor']} -
        {berikutnya['nama']}") else:
        print("Antrean Next : - (Tidak ada)") print(f"Total orang
        dalam antrean: {total_antrean}")

    def layani_pelanggan(self, menu_list):
        """ Dequeue & Transaksi: panggil pelanggan pertama terus proses orderannya. """
        console = Console(highlight=False) if
        len(self.antrian) == 0:
            console.print("\n[[bold yellow][INFO] Tidak ada pelanggan dalam antrean untuk dilayani.[/bold yellow]")
            return

        # tahan index 0 (jangan di-pop dulu, ntar aja kalo udah fix bayar) pelanggan = self.antrian[0]

        # tampilkan menu UI dari object linked list
        menu_list.tampilkan_menu()

        console.print(f"\n[[bold green]>>> MELAYANI PELANGGAN NOMOR:
        {pelanggan['nomor']} ({pelanggan['nama']} - {pelanggan['tipe']})[/bold green]")
```

```

        console.print("[italic cyan]*PENTING: Ketik nama menu sesuai dengan yang ada di daftar. Tekan ENTER
kosong jika sudah selesai memesan.*[/italic cyan]\n")

        while True: # main loop buat 1 transaksi keranjang = []
            total_belanja = 0
            # dict buat track stok sementara biar kalo cancel, stok asli tetep
aman            stok_sementara = {}

            while True: # loop pesen item per item
                pesanan = input("Kasir: 'Ingin pesan apa kak?' (Tekan ENTER untuk
selesai): ").strip()

                if pesanan == "": # kalo user cuma tekan enter brarti beres milih break

                hasil_pencarian = menu_list.cari_menu(pesanan) if hasil_pencarian is
                None:
                    console.print(f"[bold red][GAGAL] Maaf, menu '{pesanan}' tidak ditemukan. Cek
ejaanmu![/bold red]")
                    continue

                # FIX 1: Gunakan nama baku dari database sebagai key, bukan input
mentah user            nama_menu_asli = hasil_pencarian.nama

                # cek stok sementara, kalo kosong ambil dari master data (database stok_saat_ini =
node)            stok_sementara.get(nama_menu_asli,

            hasil_pencarian.stok)
            harga_menu = hasil_pencarian.harga

            if stok_saat_ini <= 0:
                console.print(f"[bold red][GAGAL] Maaf, stok untuk '{nama_menu_asli}' sedang habis.[/bold
red]")
                continue

            try:
                jumlah_pesan = int(input(f"Berapa porsi {nama_menu_asli}? (Tersedia: {stok_saat_ini}): "))
            except ValueError:
                console.print(f"[bold red][ERROR] Masukkan harus angka![/bold
red]")
                continue

            if jumlah_pesan <= 0 or jumlah_pesan > stok_saat_ini: console.print(f"[bold red][GAGAL] Jumlah
tidak valid atau
melebihi stok.[/bold red]")
                continue

            # update stok sementara menggunakan nama asli stok_sementara[nama_menu_asli] = stok_saat_ini -
            jumlah_pesan
            subtotal = harga_menu * jumlah_pesan
            total_belanja += subtotal

            # FIX 2: Cek apakah menu sudah ada di keranjang. Jika ada, gabungkan angkanya.
            item_sudah_ada = False
            for item in keranjang:
                if item['menu'] == nama_menu_asli: item['jumlah'] += jumlah_pesan
                item['subtotal'] += subtotal
                item_sudah_ada = True
                break

            # Jika belum ada di keranjang, buat baris baru if not item_sudah_ada:
            keranjang.append({
                "menu": nama_menu_asli, "jumlah":
                jumlah_pesan,

```

```

        "harga_satuan": harga_menu, "subtotal": subtotal
    })
    console.print(f"[bold green] + Berhasil menambah {jumlah_pesana}x
{nama_menu_asli}[/bold green]\n")

    # --- FASE KONFIRMASI ---
    if not keranjang:
        console.print("[yellow][INFO] Pelanggan tidak memesan apa
pun.[/yellow]")
        self.antrian.pop(0) # pop & buang pelanggan dari antrean break

    console.print("\n[bold cyan]--- REVIEW PESANAN ---[/bold cyan]") for item in keranjang:
        subtotal_format = f"Rp {item['subtotal']:,}".replace(",", ".")
        total_format = f"Rp {total_belanja:,}".replace(",", ".")
        console.print(f"- {item['menu']} (x {item['jumlah']}) : [yellow]
{subtotal_format}[/yellow]")
    console.print(f"[bold yellow]TOTAL BAYAR: {total_format}[/bold
yellow]")

    while True:
        yakin = input("\nApakah pesanan sudah benar? (y/n):
").strip().lower()
        if yakin == 'y' or yakin == 'n': break
        console.print("[bold red]Input tidak valid! Harap ketik 'y' atau 'n'.[/bold red]")
        if yakin == 'y':
            # potong stok real di Linked List for item in
            keranjang:
                menu_list.kurangi_stok(item['menu'], item['jumlah'])

            self.antrian.pop(0) # pesanan fix, buang dari list antrean self._cetak_struk(pelanggan, keranjang,
            total_belanja) self.simpan_data_ke_txt(menu_list)
            break
        else:
            console.print("[bold yellow]Baik, mari kita ulang pesannya dari awal...[/bold yellow]\n")

def _cetak_struk(self, pelanggan, keranjang, total_belanja):
    """ Pisah logic print struk biar kodingan lebih rapi & modular. """ console = Console(highlight=False)

    # 1. Kumpulkan semua baris teks ke dalam satu variabel string (isi_struk) isi_struk = f"Pelanggan:
{pelanggan['nama']} ({pelanggan['tipe']})\n" isi_struk += "-" * 44 + "\n"

    for item in keranjang:
        # format duit ke IDR
        h_satuan = f"Rp {item['harga_satuan']:,}".replace(",", ".")
        s_total = f"Rp {item['subtotal']:,}".replace(",", ".")
        isi_struk += f"{item['menu']} (x {item['jumlah']})\n" isi_struk += f" @
{h_satuan:<15} {s_total}\n"

    isi_struk += "-" * 44 + "\n"
    total_format = f"Rp {total_belanja:,}".replace(",", ".")
    isi_struk += f"[bold green]TOTAL BAYAR: {total_format}[/bold green]\n" isi_struk += "-" * 44 + "\n"
    isi_struk += "                Terima kasih telah berkunjung!                "

    # 2. Bungkus string tadi ke dalam Panel berwarna putih struk_panel = Panel(
        isi_struk,
        title="[bold white] STRUK PEMBELIAN SARIAWAM [/bold white]", border_style="white", # Ubah
        warna_garis_panel_jadi_putih expand=False # Biar lebar panelnya pas ngikutin teks, ngga full layar
terminal
    )

```

```

# 3. Print panelnya ke layar
console.print() # Print enter kosong biar ada jarak console.print(struk_panel)
console.print()

def simpan_data_ke_txt(self, menu_list, nama_file="data/menu.txt"): """ Rewrite database txt tiap
kali beres transaksi. """
    console=Console(highlight=False) try:
        # narik semua string data terbaru dari method Linked List data_menu_terupdate =
        menu_list.ambil_semua_data()

        # overwrite isi file txt-nya (w flag)
        with open(nama_file, 'w', encoding='utf-8') as file: for item in
            data_menu_terupdate:
                file.write(item + "\n")

    except Exception as e:
        console.print(f"[[red]]SISTEM ERROR[/red]] Terjadi kesalahan saat menyimpan file: {e}")

```